LocalLLM-DataForge: A Modular Pipeline for Synthetic User Story Decomposition Datasets with LLM-as-a-Judge Validation

Angel Luis Valdés Sánchez[1],
Carlos Alberto Fernández y Fernández[2*],
Verónica Rodríguez López[2]

[1]División de Estudios de Posgrado, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapan de León, 69004, Oaxaca, México.
[2*]Instituto de Computación, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapan de León, 69004, Oaxaca, México.

*Corresponding author(s). E-mail(s): caff@gs.utm.mx;
Contributing authors: alvaldes.dev@gmail.com; veromix@gs.utm.mx;

Abstract

Breaking user stories into concrete development tasks is a routine part of agile projects, but one that relies heavily on human judgment. While automating this step with machine learning is appealing, it often runs into two practical issues: a lack of annotated data and the cost of cloud-based LLMs. This paper introduces LocalLLM-DataForge, a modular, DataFrame-based framework for generating synthetic datasets using locally executed LLMs. The system runs through Ollama, includes an intelligent caching layer to avoid redundant computation, and applies automated quality checks inspired by LLM-as-a-Judge approaches. Two evaluation strategies are explored: one assesses a single generator against five explicit criteria, while the other compares outputs from two generators and selects the stronger decomposition. Experiments on the Salony dataset with models such as Llama 3.1 and Mistral suggest that caching significantly shortens experimentation cycles and that the resulting task decompositions remain logically consistent and broadly aligned with expert judgment. Overall, the framework offers a reproducible approach for requirements engineering without dependence on cloud services.

Keywords: Requirements engineering, Natural language processing, Large language models, Synthetic data generation, Task decomposition, Automated evaluation

1 Introduction

User stories are one of the most widely used requirement artifacts in agile software development. Teams commonly rely on them to capture functional needs from the end-user perspective in a concise and accessible manner [1, 2]. In practice, their role goes beyond communication: user stories are routinely used to support planning, effort estimation, and day-to-day development activities. In particular, the decomposition of stories into concrete development tasks becomes critical during backlog refinement and sprint planning [3, 4]. Even so, this activity is still largely driven by manual effort and accumulated team experience.

When task decomposition is handled manually, a number of recurring issues tend to appear. Different team members may interpret the same user story in slightly different ways, leading to inconsistent task granularity and implicit assumptions that are never fully documented. Important details can be overlooked, while other aspects are broken down too finely. Over time, these inconsistencies contribute to poorly structured backlogs that complicate estimation and task assignment [5]. The problem is often amplified in large or distributed teams, where the absence of shared decomposition practices weakens traceability and makes systematic automation in requirements engineering workflows more difficult [6].

Recent advances in Natural Language Processing (NLP), together with the rapid evolution of Large Language Models (LLMs), have opened new possibilities for automating tasks that were traditionally handled by software engineers [7, 8]. Prior studies have explored the use of LLMs for requirement classification, ambiguity detection, test case generation, and the derivation of development tasks from natural language descriptions [9, 10]. However, most existing solutions rely on proprietary, cloud-based models. This reliance introduces practical concerns related to cost, data confidentiality, reproducibility, and long-term availability, both in research settings and industrial environments.

A common limitation across many LLM-based approaches is the lack of suitable datasets for training and evaluation. Public datasets focused specifically on user story decomposition remain scarce, largely due to the high cost of expert annotation and the sensitive nature of real-world requirements [11]. Synthetic data generation using LLMs has been proposed as a partial workaround. Still, many existing methods provide limited insight into how generated outputs are validated. As a result, assessing semantic correctness, completeness, or logical consistency becomes challenging [12]. Uncontrolled generation also increases the risk of hallucinations, which can undermine trust in the resulting datasets.

This paper introduces LocalLLM-DataForge, a modular framework designed to generate synthetic datasets for user story decomposition using locally executed LLMs. The framework is built around DataFrames to structure and manage intermediate results. It also integrates with Ollama to run modern open-source language models

without relying on cloud services. One of its central components is an intelligent caching mechanism, which avoids repeated computations during iterative experiments and helps reduce execution time when exploring different generation settings.

Quality control is addressed through automated validation mechanisms. LocalLLM-DataForge applies LLM-as-a-Judge strategies to assess the generated task decompositions. Two evaluation approaches are considered. The first relies on a single generator and evaluates its outputs against explicit quality criteria aligned with ISO/IEC/IEEE 29148:2018 standards: coherence, completeness, feasibility, format, and granularity [13]. The second compares outputs produced by multiple generators and selects the most suitable decomposition. This comparative strategy leverages consensus principles known to improve reliability and reduce semantic errors in model outputs [14, 15].

This work makes three main contributions. First, it presents a fully local, extensible, and reproducible framework for generating synthetic datasets from user stories, explicitly addressing cost and privacy constraints. Second, it defines and operationalizes automated LLM-as-a-Judge validation strategies tailored to requirements engineering artifacts. Third, it reports empirical results on the Salony dataset using several open-source LLMs, highlighting both efficiency gains from caching and quality improvements obtained through comparative model selection.

The remainder of this paper is structured as follows. Section 2 reviews related work on LLM-based automation in requirements engineering and synthetic data generation. Section 3 describes the architecture and core components of LocalLLM-DataForge. Section 4 presents the experimental setup and evaluation methodology. Section 5 discusses the results, while Section 6 outlines conclusions and directions for future research.

2 Related Work

3 LocalLLM-DataForge Framework

4 Methods

The LocalLLM-DataForge tool was employed to design and execute an exploratory experimental study aimed at evaluating the feasibility of automatic synthetic decomposition of user stories using locally executed language models. The experimental environment consisted of an Apple computer equipped with an M2 chip (8 CPU cores), 16 GB of RAM, and running Python v3.13.7. Ollama v0.16.1 was used as the orchestration platform for the local execution of LLMs. All processing related to synthetic data generation and subsequent evaluation was performed entirely on this local hardware, without relying on cloud services. This setup ensured reproducibility, full control over computational resources, and confidentiality of the processed data.

The base dataset employed was Salony[1], which originally contains 1,999 user stories. In order to ensure structural consistency and avoid ambiguities derived from

[1]Salony: User Story Dataset available on Hugging Face, https://huggingface.co/datasets/salony/User_story.

heterogeneous formats, a preprocessing step was conducted in which only stories strictly conforming to the canonical structure *"As a(n) [role], I want [feature] so that [benefit]"* were retained. After this filtering process, the final dataset consisted of 1,139 valid user stories. These stories constituted the initial DataFrame of the pipeline and were used exclusively for synthetic generation purposes. No train/test split was performed, as the objective of the study was not to train predictive models but to evaluate the generative capability and methodological feasibility of the proposed approach.

Once the experimental dataset was defined, the processing architecture was designed. This architecture was implemented using the `DataForgePipeline` class, which enables chaining specialized steps under a modular DataFrame-based design. In the first stage, the `LoadDataFrame` component loaded the preprocessed dataset into the processing pipeline. Subsequently, two independent instances of the `OllamaLLMStep` component were configured, corresponding to two differentiated generators. Generator A used the `llama3.1:8b` model with a `batch_size` of 2, a temperature of 0.3 to favor more deterministic behavior, and a maximum output length of 1,000 tokens (`num_predict`). Generator B employed the `qwen3:8b` model, also with `batch_size` of 2 and a 1,000-token limit, but with a temperature of 0.7 to promote greater creativity in task generation. These configurations were selected to contrast a more deterministic model against one with higher generative variability, enabling qualitative observation of differences in decomposition patterns.

Each generator produced an independent list of tasks for every user story, with results stored in separate DataFrame columns. Since each story generated two independent decompositions, a systematic comparative evaluation mechanism was required. This stage was implemented through the `ComparisonJudgeStep` component, which applies the LLM-as-a-Judge paradigm to contrast both outputs. This approach consists of employing a language model not as a generator, but as a structured evaluator of artifacts produced by other models. The paradigm has been increasingly adopted in experimental research contexts where traditional automatic metrics are insufficient to capture semantic quality, structural coherence, or contextual adequacy. In this study, the evaluator model (`llama3.1:8b`) was configured with a reduced temperature (0.2) and a `batch_size` of 1 to maximize judgment consistency and minimize stochastic variability. Its role was not to replace human evaluation, but to provide a systematic, reproducible, and scalable mechanism to assess the quality of generated decompositions under explicitly defined criteria.

The experimental design can be characterized as an exploratory feasibility study in which the independent variable was the generator configuration (model and temperature), while the primary dependent variable was the total quality score assigned by the evaluator model. Multiple experimental runs were conducted with variations in prompt formulations and parameter settings in order to observe consistency across results and analyze model behavior under different controlled configurations.

Model calls were processed using a configurable batch processing scheme to optimize memory usage and prevent context window overflows imposed by local models. Batch size was determined considering hardware memory constraints and the potential length of generated outputs. Additionally, the internal caching mechanism of the pipeline was enabled through an optional parameter, preventing recomputation when

inputs and configurations remained unchanged. This strategy reduced experimentation time and ensured exact reproducibility across successive executions. The complete pipeline was defined in reproducible scripts within the project repository, such as `examples/salonypipeline.py` and `examples/salonydualgeneratorpipeline.py`, which can be executed directly from the command line using Python.

Once the technical configuration of the pipeline and its execution mechanisms were described, it is essential to detail the evaluation scheme adopted in the study. For each user story, five evaluation criteria were considered: coherence, completeness, feasibility, format, and granularity. Each criterion was scored on a scale from 0 to 10, resulting in a maximum total score of 50 points. The selection and definition of these criteria is grounded in the quality principles for requirements established by the ISO/IEC/IEEE 29148:2018 standard [**?**], which emphasizes characteristics such as clarity, consistency, completeness, and verifiability of requirements artifacts. In this sense, coherence evaluated the semantic correspondence between the generated tasks and the original user story, aligning with the consistency property. Completeness measured the degree of coverage of implicit and explicit requirements, in accordance with the completeness attribute defined by the standard. Feasibility analyzed whether the proposed tasks were technically realizable, which relates to the verifiability and feasibility of requirements. The format criterion verified the structure and clarity of the output, in line with the need for requirements to be understandable and unambiguous. Finally, granularity assessed the appropriateness of the decomposition level, supporting the proper specification and atomicity recommended in requirements engineering.

Coherence evaluated the semantic correspondence between the generated tasks and the original user story, aligning with the consistency property emphasized in ISO/IEC/IEEE 29148:2018. Completeness measured the degree of coverage of implicit and explicit requirements, in accordance with the completeness attribute defined by the standard. Feasibility analyzed whether the proposed tasks were technically realizable, relating to the verifiability and feasibility of requirements. The format criterion verified the structure and clarity of the output, in line with the need for requirements to be understandable and unambiguous. Granularity assessed the appropriateness of the decomposition level, supporting the proper specification and atomicity recommended in requirements engineering.

This multidimensional evaluation avoided simplified binary judgments and provided a richer representation of model performance. Such an approach is particularly relevant in contexts where quality cannot be reduced to lexical similarity but depends on conceptual and structural adequacy.

Furthermore, prompt design constituted a key methodological component of the experiment. The generation prompt was structured under an instruction–input–response schema, explicitly requiring the story to be decomposed into unique, actionable, and non-overlapping tasks while enforcing a rigid format based on sequential numbering and mandatory summary and description fields. This structural constraint aimed to reduce output variability and facilitate subsequent automated processing within the DataFrame. Similarly, the evaluator prompt was designed to operate as a multidimensional validation mechanism, explicitly defining the five evaluation criteria and establishing critical rejection rules in cases of threshold non-compliance or

severe structural errors. The judge was required to output strictly valid JSON, without additional text, to guarantee reliable programmatic extraction of generated metrics.

An approval threshold of 35 points (70% of the maximum possible score) was established as the minimum acceptability criterion, ensuring that a decomposition satisfied a substantial proportion of the defined quality attributes. The evaluator model automatically generated partial scores and total scores for each instance, recording a boolean approval indicator. When the score fell below the defined threshold, the system additionally stored a list of identified issues and improvement recommendations. In parallel, expert human evaluators reviewed a representative sample of decompositions to contrast the consistency of automated judgments and determine the reliability of the LLM-as-a-Judge approach. Within this hybrid scheme, human evaluation was considered the definitive validation criterion, while the evaluator model functioned as a structured and scalable support mechanism.

Evaluation results were stored in specific DataFrame columns, such as `validacion_coherencia`, `validacion_completitud`, `validacion_viabilidad`, `validacion_formato`, `validacion_granularidad`, `validacion_total`, and `validacion_aprobado`, enabling systematic subsequent analysis.

Overall, the methodology was designed to be modular, reproducible, and executable in local environments with limited resources. The combination of step-based architecture, batch processing, explicit hyperparameter configuration, and caching mechanisms ensures full traceability of the experimental process and allows replication of results under identical configuration and software version conditions, thereby consolidating the technical validity of the presented feasibility study.

5 Discussion

6 Conclusion

Acknowledgements.

Declarations

- Funding: Not applicable
- Conflict of interest: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable
- Consent for publication: Not applicable
- Data availability: The datasets and pipeline code are available at the project repository.
- Code availability: The source code for LocalLLM-DataForge is publicly available.
- Author contribution: All authors contributed to the design and development of the framework. A.L.V.S. led implementation and experiments. C.A.F.F. and V.R.L. supervised the research and contributed to methodology.

References

[1] Cohn, M.: User Stories Applied: For Agile Software Development. Addison-Wesley, ??? (2004)

[2] Beck, K.: Extreme Programming Explained: Embrace Change. Addison-Wesley, ??? (2001)

[3] Rubin, K.S.: Essential Scrum: A Practical Guide to the Most Popular Agile Process. Addison-Wesley, ??? (2012)

[4] Schwaber, K., Sutherland, J.: The Scrum Guide. https://scrumguides.org. Accessed: 2026-03-02 (2020)

[5] Lucassen, G., Dalpiaz, F., Werf, J.M.E., Brinkkemper, S.: Improving agile requirements: the quality user story framework and tool. Requirements Engineering **21**(3), 383–403 (2016) https://doi.org/10.1007/s00766-016-0250-x

[6] Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. IEEE Software **23**(6), 88–95 (2006) https://doi.org/10.1109/MS.2006.147

[7] Chen, M., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

[8] White, J., Fu, Q., Hays, S., Sandborn, M., Oleynik, V., et al.: A prompt pattern catalog to enhance prompt engineering with large language models. arXiv preprint arXiv:2302.11382 (2023)

[9] Rahimi, M., Gervasi, V.: Natural language processing for requirements engineering: A systematic mapping study. IEEE Access **11**, 25130–25152 (2023) https://doi.org/10.1109/ACCESS.2023.3251234

[10] Sebban, M., et al.: Large language models for software requirements engineering: Opportunities and challenges. Empirical Software Engineering (2024)

[11] Fucci, D., Palomares, C., Franch, X.: Data-driven requirements engineering: Models and methods. Requirements Engineering **25**, 1–24 (2020) https://doi.org/10.1007/s00766-019-00315-3

[12] Liu, Y., et al.: Synthetic data generation with large language models: Challenges and opportunities. arXiv preprint arXiv:2305.12867 (2023)

[13] Chiang, D., Lee, J.-U.: Large language models are reasoning teachers. arXiv preprint arXiv:2305.08328 (2023)

[14] Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2023)

[15] Li, Y., et al.: Consensus-based evaluation of large language model outputs. arXiv preprint arXiv:2401.01335 (2024)